KINGA AutoVerify AI — Technical Architecture Document

Classification: Confidential — For Technical Investors, Senior Engineers, Insurance Executives, and Forensic Audit Teams **Version:** 1.0 | **Date:** April 2026 **Platform Version:** See `KINGA_PLATFORM_VERSION` in `server/pipeline-v2/felVersionRegistry.ts`

Table of Contents

1. System Overview
2. End-to-End Pipeline Architecture
3. Data Flow and State Management
4. Reconciliation and Decision Engine
5. AI and Model Usage
6. Fraud Detection and Risk Scoring
7. Forensic Engine — Core Differentiator
8. System Reliability and Failure Handling
9. UI and Reporting Layer
10. Integrations and Extensibility
11. Security and Governance
12. Performance and Scalability
13. Limitations and Future Improvements
14. Executive Summary

---

1. System Overview

1.1 Core Purpose

KINGA AutoVerify AI is an end-to-end motor insurance claims analysis and forensic assessment platform. It ingests raw claim documents — PDFs, scanned forms, repair quotes, police reports, and damage photographs — and produces a structured Forensic Audit Report within minutes. The report covers accident physics reconstruction, multi-source damage analysis, repair cost

benchmarking, fraud risk scoring, and a decision recommendation (APPROVE / REVIEW / REJECT / NEGOTIATE / ESCALATE).

The platform is designed for deployment by motor insurers, third-party administrators, and claims management companies operating in markets where manual assessment is slow, expensive, and inconsistent. Its primary deployment context is Zimbabwe, where USD-denominated repair costs, cross-border repair scenarios (South Africa), and a high volume of imported second-hand vehicles create a distinctive set of fraud vectors and cost benchmarking challenges.

1.2 Key Problems Solved

Problem	KINGA's Response
Manual assessors cost 25–50 per claim and introduce subjective variance	Automated pipeline produces a consistent, auditable report at 5–12 per claim
Fraud detection relies on adjuster intuition	10+ quantified fraud signals with explainable scores and evidence citations
Repair quotes are accepted without independent benchmarking	AI cost engine cross-validates quotes against component-level market rates
Speed and energy claims are unverifiable from photos alone	Physics engine applies five independent methods (Campbell, Impulse, Vision, Deployment Threshold) to reconstruct impact speed
No audit trail for AI-assisted decisions	Forensic Execution Ledger (FEL) records every stage's input hash, output snapshot, model version, and assumptions
Claims processing takes days	Pipeline completes in under 3 minutes for a standard 5-document claim

1.3 System Philosophy

Deterministic over probabilistic where physics applies. The speed inference ensemble, crush depth estimation, and kinetic energy calculations are pure mathematics — zero LLM calls, zero stochastic variance. A given set of inputs always produces the same physics output. This is a deliberate design choice: forensic evidence must be reproducible and defensible in a dispute or legal proceeding.

Probabilistic where language understanding is required. Document OCR, damage description parsing, fraud narrative analysis, and incident classification use large language models. These components are wrapped in structured output schemas (JSON Schema enforcement), confidence scoring, and cross-validation against deterministic outputs to contain hallucination risk.

Never halt. The pipeline is designed to produce a report for every claim, regardless of document quality. Every stage implements a fallback path. Missing data is tracked explicitly in an `Assumption`

registry and surfaced to the adjuster. A claim with poor documentation produces a degraded report with low confidence scores — not a processing failure.

Full auditability. Every conclusion is traceable to its source evidence, the engine that produced it, the model version used, and the assumptions introduced. The Forensic Execution Ledger (FEL) and Evidence Registry are designed to satisfy court-grade audit requirements.

1.4 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        KINGA Platform                                 │
│                                                                       │
│  ┌──────────┐    ┌──────────────────────────────────────────────┐   │
│  │  Intake  │    │           Self-Healing Pipeline               │   │
│  │  Layer   │──▶│  S1 → S2 → S3 → S4 → S5 → S6 → S7 → S8 → S9 │──▶│
│  │(Web/API) │    │                      ↓                        │   │
│  └──────────┘    │               Stage 10 Report                 │   │
│                  └──────────────────────────────────────────────┘   │
│                                       │                              │
│  ┌────────────────────────────────────▼──────────────────────────┐  │
│  │              Post-Pipeline Validation Layer                    │  │
│  │  Causal Chain | Evidence Bundle | Realism Bundle | Consensus|  │
│  │  FEL Snapshot | Case Signature  | Decision Authority          |  │
│  └────────────────────────────────────────────────────────────────┘  │
│                                       │                              │
│  ┌────────────────────────────────────▼──────────────────────────┐  │
│  │                     Persistence Layer                          │  │
│  │          MySQL/TiDB (claims, ai_assessments, tenants)          │  │
│  │          S3 (documents, photos, generated reports)             │  │
│  └────────────────────────────────────────────────────────────────┘  │
│                                       │                              │
│  ┌────────────────────────────────────▼──────────────────────────┐  │
│  │                     Presentation Layer                         │  │
│  │   Adjuster Dashboard | Forensic Audit Report | Admin Panel     │  │
│  └────────────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────────┘
```

The platform is built on a React 19 + TypeScript frontend, an Express 4 + tRPC 11 backend, and a MySQL/TiDB database. All AI calls are routed through server-side helpers (`invokeLLM`, `generateImage`, `transcribeAudio`) that use the Manus Built-in Forge API, ensuring API keys are never exposed to the client.

---

2. End-to-End Pipeline Architecture

The pipeline is implemented in `server/pipeline-v2/orchestrator.ts`. It is invoked by `runPipelineV2(ctx: PipelineContext)` and **never throws** — every stage either succeeds, degrades gracefully, or produces a documented fallback output. The orchestrator collects `Assumption[]` and `RecoveryAction[]` from every stage and passes them to Stage 10 for inclusion in the final report.

A `PipelineStateMachine` tracks execution state and enforces allowed transitions. Critical stages (`1_ingestion`, `2_extraction`, `3_structured_extraction`, `5_assembly`) trigger a `FLAGGED_EXCEPTION` state if they fail, which is surfaced in the dashboard but does not halt processing.

Each stage is wrapped in `runWithTimeout(stageKey, fn)` which enforces per-stage time budgets defined in `pipelineContractRegistry.ts`. A `StageTimeoutError` is caught and converted to a degraded result with an assumption record.

Stage 0 — Evidence Registry Bootstrap

Objective: Establish a structured evidence registry before any extraction begins, so that every piece of evidence can be linked to a source document and a confidence level throughout the pipeline.

Inputs: `PipelineContext` (claim ID, uploaded document URLs, claim DB record)

Outputs: `EvidenceRegistry` — a typed map of evidence categories (vehicle identity, incident narrative, damage, financial, police) to their source documents and confidence scores.

Processing Logic: `buildEvidenceRegistry()` in `evidenceRegistryEngine.ts` inspects the document list and pre-populates the registry with `PENDING` entries. As later stages extract data, they update the registry with `CONFIRMED` or `INFERRED` status and a confidence score (0–100).

Failure Mode: If the registry cannot be built (no documents), it is initialised as an empty object. All downstream stages treat a missing registry as `PENDING` evidence for all categories.

Stage 1 — Document Ingestion

Objective: Classify, normalise, and index all uploaded documents (PDFs, images, Word files) into a typed `IngestedDocument[]` array.

Inputs: Raw file URLs from S3 (uploaded by the adjuster or claimant via the web interface).

Outputs: `Stage1Output` — `{ documents: IngestedDocument[], primaryDocumentIndex: number, totalDocuments: number }`

Processing Logic:

- Each file is fetched from S3 and classified by MIME type and filename heuristics into one of: `claim_form`, `police_report`, `repair_quote`, `vehicle_photos`, `supporting_document`, `unknown`.
- PDF files are passed through `pdftoppm` (server-side utility) to extract embedded images as separate URLs, which are stored in `IngestedDocument.imageUrls`.
- The `primaryDocumentIndex` identifies the claim form (the document most likely to contain structured claim data).

AI Models Used: None — classification is heuristic (filename patterns, MIME type, page count).

Confidence Scoring: Each document receives a `containsImages: boolean` flag. Documents with images are prioritised for Stage 6 (Damage Analysis).

Failure Mode: If Stage 1 times out or fails, the orchestrator produces an empty document set (`documents: []`) and continues. All downstream stages fall back to the claim's database fields (pre-populated at claim submission time).

Dependencies: None — first stage.

Stage 2 — OCR and Text Extraction

Objective: Extract all readable text from every ingested document, including scanned PDFs and image-based documents.

Inputs: `Stage1Output.documents[]`

Outputs: `Stage2Output` — `{ extractedTexts: ExtractedText[], totalPagesProcessed: number }` where each `ExtractedText` contains `rawText`, `tables: ExtractedTable[]`, `ocrApplied: boolean`, and `ocrConfidence: number` (0–100).

Processing Logic:

- Native PDF text is extracted directly via PDF parsing libraries.
- Image-based pages (scanned documents) are sent to the LLM vision API with a structured OCR prompt requesting verbatim text extraction and table preservation.
- Structured tables (repair quote line items, parts lists) are extracted as `ExtractedTable[]` with `headers[]` and `rows[][]` for downstream structured parsing.
- Raw OCR text is persisted to `ai_assessments.stage2RawOcrText` for audit and re-extraction without re-running the full pipeline.

AI Models Used: Vision LLM (for scanned pages) — structured output mode, no hallucination risk for verbatim text extraction.

Confidence Scoring: `ocrConfidence` is set to 90 for native PDF text, 60–80 for vision-extracted text depending on image quality.

Failure Mode: If OCR fails for a document, that document's `rawText` is set to an empty string and a `DOCUMENT_LIMITATION` assumption is recorded. Downstream stages use claim database fields as the primary source.

Stage 3 — Structured Data Extraction

Objective: Parse the raw OCR text from all documents into a typed `ExtractedClaimFields` schema — the canonical set of 50+ structured fields covering vehicle identity, incident details, police report data, repair quote financials, and damage components.

Inputs: `Stage2Output.extractedTexts[]`

Outputs: `Stage3Output` — `{ perDocumentExtractions: ExtractedClaimFields[], inputRecovery: InputRecoveryOutput }`

Processing Logic:

The extraction runs in two passes:

Pass 1 — LLM Structured Extraction: Each document's raw text is sent to the LLM with a JSON Schema-enforced prompt requesting all 50+ fields. Missing fields are explicitly returned as `null` — the model is instructed never to guess. The prompt includes field-level descriptions and examples drawn from Zimbabwean claim form conventions (e.g., ZRP charge numbers, ZINARA registration formats).

Pass 2 — Input Recovery (5-Step): After LLM extraction, a deterministic recovery pass attempts to fill gaps:

1. Accident description recovery from raw text using regex patterns for common narrative structures.
2. Quote figure recovery using regex fallback (e.g., `TOTAL: USD 4,250.00` patterns) when the LLM missed financial figures.
3. Image presence detection — confirms whether damage photos are available.
4. Damage keyword extraction — builds a `DamageHints` object (`zones[]` , `components[]`) for Stage 6 fallback.
5. Failure flag assignment — sets `InputRecoveryFailureFlag[]` (`ocr_failure` , `quote_not_mapped` , `description_not_mapped` , `images_not_processed`) for downstream consumers.

The LLM Quote Engine (Step 2b) runs separately when multiple quote blocks are detected in the text, extracting one `ExtractedQuoteRecord` per panel beater for multi-quote optimisation in Stage

9b.

Key Extracted Fields:

Category	Fields
Vehicle	make, model, year, VIN, registration, colour, engine number, mileage
Incident	date, location, description, incident type, collision direction, speed (km/h), impact point, airbag deployment, crush depth (m), damage area (m^2)
Police	report number, station, officer name, charge number, fine amount, investigation status, officer findings, third-party account
Financial	quote total, agreed cost, labour cost, parts cost, market value, excess, betterment
Insurance	insurer name, policy number, product type, claim reference
Third Party	driver name, vehicle description, registration, insurer, policy number

AI Models Used: LLM (structured JSON output with JSON Schema enforcement). Quote Engine uses a separate LLM call with a quote-specific schema.

Hallucination Mitigation: JSON Schema enforcement with `additionalProperties: false` and `required` arrays. The model is explicitly instructed to return `null` for missing fields rather than infer values. All non-null values are cross-validated in Stage 4.

Failure Mode: If LLM extraction fails for a document, that document's fields are all set to `null`. The Input Recovery pass still runs on the raw text. A `SYSTEM_EXTRACTION_FAILURE` assumption is recorded.

Stage 4 — Data Validation and Field Arbitration

Objective: Merge extractions from multiple documents into a single validated `ExtractedClaimFields`, resolve conflicts between documents, score data completeness, and run a pre-analysis consistency check.

Inputs: `Stage3Output.perDocumentExtractions[]`

Outputs: `Stage4Output — { validatedFields, issues: ValidationIssue[], completenessScore: number, missingFields: string[], fieldValidation, consistencyCheck, gateDecision }`

Processing Logic:

Source-Priority Arbitration: For the four most critical fields (quote total, collision direction, vehicle make/model, incident type), a `FieldValidationEngine` applies a priority hierarchy: police report > claim form > repair quote > supporting documents. Conflicts between sources are flagged as `ValidationIssue` with `severity: "warning"` or `"critical"`.

Completeness Scoring: Each of the 50+ fields is weighted by its importance to downstream analysis. The `completenessScore` (0–100) is the weighted proportion of non-null fields. A score below 40 triggers a `DEGRADED_DATA` flag.

Pre-Analysis Consistency Check: `claimConsistencyChecker.ts` runs 12 cross-field checks before any AI analysis begins:

- Date coherence (accident date before claim date, before quote date)
- Vehicle identity coherence (make/model/year plausibility)
- Financial coherence (parts + labour ≈ total within 5%)
- Incident type coherence (airbag deployment consistent with collision type)

Pipeline Gate Controller: `pipelineGateController.ts` evaluates the consistency check result and sets `gateDecision.status` to `"PROCEED"`, `"DEGRADE"`, or `"HOLD"`. A `"HOLD"` decision means the pipeline must not proceed to analytical stages — this occurs when critical identity fields (vehicle registration, claim ID) are missing and cannot be recovered.

Failure Mode: If Stage 4 fails, the pipeline uses the raw Stage 3 output directly. All validation issues are marked as unresolved.

Stage 5 — Claim Record Assembly

Objective: Assemble the validated fields into a `ClaimRecord` — the single canonical data object passed to all analysis engines (Stages 6–9). Apply automotive domain corrections, infer vehicle mass, classify the collision scenario, and trigger vehicle market valuation.

Inputs: `Stage4Output.validatedFields`

Outputs: `Stage5Output` — `{ claimRecord: ClaimRecord, scenarioSelection }`

Processing Logic:

ClaimRecord Assembly: All validated fields are mapped into the typed `ClaimRecord` structure, which groups data into: `vehicle: VehicleRecord`, `driver: DriverRecord`, `accidentDetails: AccidentDetails`, `policeReport: PoliceReportRecord`, `damage: DamageRecord`, `repairQuote: RepairQuoteRecord`, `insuranceContext`, and `dataQuality`.

Automotive Domain Corrections: `automotiveDomainCorrector.ts` applies 200+ correction rules for common OCR errors and regional naming conventions in the Zimbabwean market:

- Make/model normalisation (e.g., “TOYOTA HILUX SURF” → make: “Toyota”, model: “Hilux Surf”)
- Body type inference from model name (e.g., “Hilux” → “pickup”)
- Powertrain inference (e.g., “D4D” suffix → diesel ICE)

Vehicle Mass Inference: If the claim form does not state the vehicle mass, the system uses a three-tier lookup:

1. Model-specific lookup table (e.g., Toyota Hilux 2.4 GD-6 → 1,905 kg)
2. Body-type class average (e.g., SUV → 1,800 kg)
3. Global default (1,400 kg) — recorded as a `MARKET_DEFAULT` assumption with confidence 40.

The `massTier` field (`explicit | inferred_model | inferred_class | not_available`) is stored in `VehicleRecord` and surfaced in the Forensic Audit Report.

Collision Scenario Classification: The `CollisionScenario` type classifies the incident into one of 12 scenarios (e.g., `rear_end_struck`, `head_on_collision`, `single_vehicle_rollover`, `animal_strike`). This drives physics routing, evidence requirements, and fraud detector profiles in Stages 7 and 8.

Multi-Event Sequence Detection: `detectMultiEventSequence()` parses the incident narrative for compound events (e.g., “vehicle was struck from behind, causing it to veer off the road and roll”). When detected, an `IncidentEvent[]` sequence is stored in `AccidentDetails.multiEventSequence`, and each event is mapped to its damage contribution zones.

Vehicle Market Valuation (Stage 5b): `valuateVehicle()` queries AutoTrader ZA data and LLM-based market estimates to produce a `VehicleValuation` with `marketValueUsd`, `repairToValueRatio`, and a `verdict` (`repairable | write_off | borderline | unknown`). If the repair cost exceeds 70% of market value, a total-loss flag is set.

Failure Mode: If assembly fails, the orchestrator constructs a minimal `ClaimRecord` from the claim’s database fields directly. All assembly assumptions are recorded.

Stage 6 — Damage Analysis Engine

Objective: Analyse all damage photographs using computer vision to produce a structured list of damaged components with severity scores, damage zones, structural damage detection, and total damage area.

Inputs: `ClaimRecord.damage.imageUrls[]`, `ClaimRecord.accidentDetails.collisionDirection`

Outputs: `Stage6Output` — `{ damagedParts: DamageAnalysisComponent[], damageZones: DamageZone[], overallSeverityScore: number, structuralDamageDetected: boolean, totalDamageArea: number, photosAvailable, photosProcessed, photosDeferred, photosFailed, imageConfidenceScore, perPhotoResults[] }`

Processing Logic:

Each photo URL is processed independently by the vision LLM with a structured damage analysis prompt. The prompt requests:

- A list of damaged components with `name`, `location`, `damageType` (`dent | crack | scratch | deformation | missing | broken`), `severity` (`cosmetic | minor | moderate | severe | catastrophic`), and `distanceFromImpact` (cm).
- Whether structural damage (chassis, frame, A/B/C pillars) is visible.
- An estimated damage area (m^2) for the visible zone.
- A vision-extracted crush depth (cm) — used as M5 input for the speed inference ensemble.

Direction-Aware Filtering: Components reported by the vision model that are inconsistent with the stated collision direction are filtered out and logged in `excludedComponents[]` for audit. For example, if the collision direction is `frontal`, rear bumper damage reported by vision is excluded unless the narrative explains a secondary impact.

Image Intelligence Layer: Each photo receives a `damageLikelihoodScore` (0–100) from a pre-filter that detects whether the image actually shows vehicle damage (vs. a document scan, a landscape photo, or an unrelated image). Photos scoring below 20 are marked `SKIPPED_BUDGET` rather than sent to the full vision analysis.

Photo Accounting: The output includes an honest accounting of every photo: `photosAvailable` (total), `photosProcessed` (sent to vision LLM), `photosDeferred` (budget limit), `photosFailed` (LLM error or HTTP 4xx/5xx). No photo may be silently omitted — every URL appears in `perPhotoResults[]` with a `status` field.

Text Fallback: If no photos are available or all photos fail, Stage 6 falls back to the damage description text from Stage 3, using keyword extraction to populate a minimal `damagedParts[]` list. `analysisFromPhotos` is set to `false` and `imageConfidenceScore` to 0.

AI Models Used: Vision LLM (multimodal) — structured JSON output with component-level schema enforcement.

Confidence Scoring: `imageConfidenceScore` (0–100) is the weighted average of per-photo confidence scores, adjusted for photo count and failure rate.

Failure Mode: If Stage 6 fails entirely, `buildDamageFallback()` produces a minimal damage output from the claim's damage description text. `overallSeverityScore` defaults to 50 (moderate) and `structuralDamageDetected` to `false`.

Stage 7 — Physics Analysis Engine

Objective: Reconstruct accident physics from the `ClaimRecord` and Stage 6 damage analysis. Compute impact force, kinetic energy, deceleration, and — most critically — estimate impact speed using a five-method ensemble. Gate execution on incident type (collision/unknown only; skip for theft, fire, flood, vandalism).

Inputs: `ClaimRecord`, `Stage6Output`

Outputs: `Stage7Output` — `{ impactForceKn, impactVector, energyDistribution, estimatedSpeedKmh, deltaVKmh, decelerationG, accidentSeverity, accidentReconstructionSummary, damageConsistencyScore, latentDamageProbability, physicsExecuted, physicsStatus, speedInferenceEnsemble, damagePatternValidation, severityConsensus }`

Processing Logic: Detailed in Section 7 (Forensic Engine). At a high level:

1. `inferCrushDepth()` computes the best available crush depth estimate from document data, damage severity, component count, structural damage, and damage area.
2. `runSpeedInferenceEnsemble()` runs up to five independent physics methods in parallel and produces a weighted consensus speed with a 90% confidence interval.
3. `validateDamagePattern()` checks whether the reported damage pattern is physically consistent with the stated collision scenario (e.g., frontal impact should produce front-zone damage, not rear-zone damage).
4. `severityConsensusEngine` fuses severity signals from physics (kinetic energy), damage analysis (component severity scores), and image analysis (vision confidence) into a single consensus severity rating.
5. `runIncidentNarrativeEngine()` (called inside `runUnifiedStage7`) performs a structured LLM analysis of the incident narrative, extracting timeline, causal chain, third-party account consistency, and narrative anomalies.

Animal Strike Physics: When `incidentType = animal_strike`, a dedicated `animalStrikePhysicsEngine` applies species-specific mass and impact geometry models (e.g., cattle at 450 kg, kudu at 120 kg) to estimate impact force and damage consistency.

Failure Mode: If the physics engine fails, `estimatePhysicsFromDamage()` produces a simplified fallback using `KE = ½mv²` with the document-stated speed (if available) or marks physics as `SKIPPED_NO_SPEED` if speed is absent. A fabricated speed is never used — the system explicitly refuses to guess speed because a guessed speed cascades errors through force, energy, cost, and fraud scoring.

Stage 7b — Causal Reasoning Engine

Objective: Produce a structured causal verdict that links the incident narrative, physics output, and damage pattern into a coherent causal chain. Identify narrative anomalies, internal contradictions, and implausibility flags.

Inputs: `ClaimRecord`, `Stage6Output`, `Stage7Output`

Outputs: `CausalVerdict — { verdict: CONSISTENT | INCONSISTENT | AMBIGUOUS, causalChain[], contradictions[], plausibilityScore, reasoning }`

Processing Logic: `runCausalReasoningEngine()` sends the incident narrative, physics summary, and damage component list to the LLM with a structured prompt requesting a causal chain analysis. The output is validated against the physics output — if the LLM's causal chain implies a speed inconsistent with the physics ensemble by more than 40%, a `HIGH_DIVERGENCE` flag is set.

Stage 8 — Fraud Analysis Engine

Objective: Score the claim for fraud risk using 10+ quantified indicators drawn from cost anomalies, damage inconsistencies, behavioral patterns, photo forensics, and cross-engine consistency checks.

Inputs: `ClaimRecord`, `Stage6Output`, `Stage7Output`

Outputs: `Stage8Output — { fraudRiskScore: number, fraudRiskLevel: FraudRiskLevel, indicators: FraudIndicator[], quoteDeviation, scenarioFraudResult, crossEngineConsistency, photoForensics }`

Processing Logic: Detailed in Section 6 (Fraud Detection). Key sub-engines:

- **Scenario-Aware Fraud Detector:** Applies scenario-specific fraud profiles (e.g., rear-end claims have different fraud vectors than single-vehicle rollovers).
- **Cross-Engine Consistency Validator:** Checks 8 named consistency pairs across physics, damage, and cost engines. Conflicts produce a `CONFLICTED` status and a `conflict_penalty` on the fraud score.
- **Photo Forensics Engine:** Analyses EXIF metadata, GPS coordinates, capture datetime, and image hash for each damage photo. Detects manipulation indicators and flags photos taken before the stated accident date.

False Positive Protection: The scenario fraud engine includes explicit false-positive suppression rules. For example, a high repair cost on a luxury vehicle does not trigger an overpricing flag if the vehicle's market value supports it. Suppressed flags are logged in `false_positive_protection[]` for transparency.

Stage 9 — Cost Optimisation Engine

Objective: Produce an independent AI cost benchmark, reconcile it against submitted quotes, identify overpricing and structural gaps, and generate a negotiation guidance package.

Inputs: `ClaimRecord`, `Stage6Output`, `Stage7Output`

Outputs: `Stage9Output` — `{ expectedRepairCostCents, quoteDeviationPct, recommendedCostRange, savingsOpportunityCents, breakdown, repairIntelligence[], partsReconciliation[], reconciliationSummary, alignmentResult, costNarrative, costReliability, costDecision, quoteOptimisation }`

Processing Logic:

Stage 9a — Component-Level Cost Estimation: For each damaged component from Stage 6, the system estimates `partsCost`, `labourCost`, and `paintCost` using a regional market rate database (Zimbabwe USD rates, South Africa ZAR rates with PPP adjustment). The `Economic Context Engine` (Stage 9 Phase 2B) applies a Purchasing Power Parity factor and a National Cost Index to normalise costs across repair markets.

Stage 9b — Quote Optimisation Engine: When multiple quotes are submitted, each quote is scored on `coverage_ratio` (what fraction of the AI-identified damaged components are included), `structurally_complete` (whether structural components are present), and `is_outlier` (whether the total cost deviates more than 2σ from the median). A weighted baseline cost is computed from non-outlier quotes.

Stage 9c — Claims Cost Decision Engine: Produces the final `costDecision` with:

- `true_cost_usd` — the authoritative cost basis (assessor-validated if an assessor has reviewed; system-optimised otherwise)
- `deviation_analysis` — highest quote deviation, AI-vs-true percentage, quote spread
- `anomalies[]` — categorised cost anomalies (overpricing, under-quoting, misaligned components, structural gaps)
- `recommendation` — APPROVE / REVIEW / REJECT / NEGOTIATE / PROCEED_TO_ASSESSMENT / ESCALATE
- `negotiation_guidance` — target, floor, and ceiling prices with per-panel-beater reduction recommendations

Parts Reconciliation: `partsReconciliation[]` maps each AI-identified component to its quoted counterpart, computing variance percentage and flagging items as `overpriced`, `underpriced`, or `no_quote`. `reconciliationSummary` reports `missing_count` (components in AI analysis but absent from quote) and `extra_count` (quoted items not in AI analysis).

Failure Mode: If Stage 9 fails, `buildCostFallback()` produces a minimal cost output using the document-stated quote total with a `LOW` reliability score.

Stage 9b — Turnaround Time Analysis

Objective: Estimate the expected repair duration and identify bottlenecks.

Inputs: `ClaimRecord`, `Stage6Output`

Outputs: `TurnaroundTimeOutput` — `{ estimatedRepairDays, bestCaseDays, worstCaseDays, confidence, breakdown: { assessmentDays, partsSourcingDays, repairDays, paintDays, qualityCheckDays }, bottlenecks[], marketRegion }`

Processing Logic: Repair duration is estimated from component count, structural damage flag, parts sourcing complexity (OEM vs aftermarket, cross-border sourcing), and historical turnaround data for the market region. Cross-border repairs (Zimbabwe claim, South Africa repair) add 3–7 days for logistics.

Stage 10 — Report Generation

Objective: Assemble all stage outputs into a structured Forensic Audit Report with confidence scores, assumption registry, evidence trace, decision readiness assessment, and claim quality score.

Inputs: All stage outputs (S1–S9b), `allAssumptions[]`, `allRecoveryActions[]`

Outputs: `Stage10Output` — `{ claimSummary, damageAnalysis, physicsReconstruction, costOptimisation, fraudRiskIndicators, turnaroundTimeEstimate, supportingImages, fullReport, confidenceScore, assumptions[], missingDocuments[], missingFields[], evidenceTrace, decisionReadiness, consistencyCheck, claimQuality, degradationReasons[] }`

Key Sub-Components:

- **Decision Readiness Engine:** Runs 12 checks (physics executed, fraud score computed, cost benchmark available, photos processed, etc.) and sets `decision_ready: boolean`. Blocking issues are listed with resolution guidance.
- **Claim Quality Scorer:** Produces a multi-dimensional quality score (0–100) with a letter grade (A–F) across dimensions: data completeness, evidence quality, physics confidence, cost reliability, fraud signal strength. Drives the `requiresManualReview` flag.
- **Cross-Stage Consistency Engine:** Runs named contradiction checks across all stage outputs (e.g., “physics says severe impact but damage analysis says cosmetic”) and produces a `ConsistencyCheckResult` with `CONSISTENT | CONFLICTED` status.

- **Degradation Reasons:** `degradationReasons[]` lists actionable reasons why the report is degraded (e.g., "No damage photos extracted from PDF — pdftoppm unavailable in production", "Physics Reconstruction unavailable — speed not extracted from claim form"). These are surfaced directly to the adjuster.

Post-Pipeline Validation Layer

After Stage 10, the orchestrator runs a series of post-pipeline engines that do not modify the core report but produce additional audit artifacts:

Engine	Output	Purpose
`causalChainBuilder`	`CausalChainOutput`	Structured causal chain linking incident → damage → cost
`evidenceStrengthScorer`	`EvidenceBundle`	Quantified evidence strength per claim section
`outputRealismValidator`	`RealismBundle`	Checks whether all outputs are within physically plausible ranges
`benchmarkDeviationEngine`	`BenchmarkBundle`	Compares this claim against the live benchmark database
`crossEngineConsensus`	`ConsensusResult`	Final cross-engine agreement score
`caseSignatureGenerator`	`CaseSignatureOutput`	Unique fingerprint for deduplication and pattern matching
`forensicExecutionLedger`	`ForensicExecutionLedger`	Court-grade per-stage audit record
`felVersionRegistry`	`FELVersionSnapshot`	Per-stage prompt hash, model ID, input/output hash, contract version
`validatedOutcomeRecorder`	`ValidatedOutcomeResult`	Learning gate — records adjuster decisions for model calibration

---

3. Data Flow and State Management

3.1 Data Movement Through the Pipeline

Data flows through the pipeline as a series of typed stage outputs, each consumed by the next stage. The `PipelineContext` object carries cross-cutting concerns (claim ID, tenant ID, log function, stage callbacks) and is passed to every stage. Stage outputs are not stored in the context

— they are passed explicitly as function arguments to the next stage, ensuring that each stage's dependencies are visible in the function signature.

```
PipelineContext
    │
    ▼
Stage 1 → Stage1Output
    │
    ▼
Stage 2 (Stage1Output) → Stage2Output
    │
    ▼
Stage 3 (Stage2Output) → Stage3Output
    │
    ▼
Stage 4 (Stage3Output) → Stage4Output
    │
    ▼
Stage 5 (Stage4Output) → Stage5Output { claimRecord: ClaimRecord }
    │
    ├──▶ Stage 6 (ClaimRecord) → Stage6Output
    │         │
    ├──▶ Stage 7 (ClaimRecord, Stage6Output) → Stage7Output
    │         │
    ├──▶ Stage 8 (ClaimRecord, Stage6Output, Stage7Output) → Stage8Output
    │         │
    └──▶ Stage 9 (ClaimRecord, Stage6Output, Stage7Output) → Stage9Output
              │
              ▼
         Stage 10 (all outputs) → Stage10Output
```

Stages 6–9 are logically independent given the `ClaimRecord` and can be parallelised. The orchestrator currently runs them sequentially to allow each stage to inform the next (Stage 7 output feeds Stage 8 fraud scoring; Stage 8 fraud score feeds Stage 9 cost decision). The `evidenceTrace.parallelStages` field in the Stage 10 output documents which stages ran concurrently in any given pipeline run.

3.2 The ClaimRecord — Central Data Structure

The `ClaimRecord` is the single canonical data object that all analysis engines consume. It is assembled in Stage 5 and never mutated by downstream stages. Its structure is defined in `server/pipeline-v2/types.ts`:

```
interface ClaimRecord {
  claimId: number;
  tenantId: number | null;
  vehicle: VehicleRecord;          // make, model, year, VIN, mass, body type,
powertrain
  driver: DriverRecord;            // name, licence, injuries
  accidentDetails: AccidentDetails; // date, location, description, speed, direction,
airbag, crush depth
  policeReport: PoliceReportRecord; // report number, officer, charge, investigation
status
  damage: DamageRecord;            // description, components, imageUrls
  repairQuote: RepairQuoteRecord;  // repairer, total, labour, parts, line items
  insuranceContext: { ... };       // insurer, policy, excess, betterment
  dataQuality: { completenessScore, missingFields[], validationIssues[] };
  marketRegion: string;            // e.g. "ZW" (Zimbabwe)
  assumptions: Assumption[];       // all assumptions made during assembly
  evidenceRegistry?: EvidenceRegistry;
  thirdParty?: ThirdPartyRecord;
  valuation?: VehicleValuation;
}
```

3.3 Persistence Layer

All pipeline outputs are persisted to the `ai_assessments` table in MySQL/TiDB. Each column stores either a scalar value or a JSON blob:

Column	Content
`fraudRiskScore`, `fraudRiskLevel`	Scalar fraud score and level
`recommendation`	Final pipeline recommendation (APPROVE/REVIEW/REJECT/etc.)
`physicsAnalysis`	Stage 7 output JSON
`costIntelligenceJson`	Stage 9 output JSON (cost decision, negotiation guidance)
`fraudScoreBreakdownJson`	Stage 8 detailed fraud indicator breakdown
`claimRecordJson`	Full ClaimRecord JSON (canonical extraction result)
`forensicExecutionLedgerJson`	Court-grade per-stage audit record
`assumptionRegistryJson`	All assumptions classified by type and impact
`pipelineRunSummary`	Per-stage status, duration, error, assumption count
`fcdiScore`	Forensic Confidence Degradation Index (0–100)
`ifeResultJson`	Input Fidelity Engine result (4-class data attribution)
`doeResultJson`	Decision Optimisation Engine result

Document files and damage photos are stored in S3. The database stores only URLs and metadata.

3.4 Idempotency and Reprocessing

Claims can be reprocessed at any time. Each pipeline run creates a new `ai_assessments` row with `isReanalysis: 1`, `previousAssessmentId`, `reanalysisReason`, and an incremented `versionNumber`. The previous assessment is preserved for audit comparison.

The `caseSignatureGenerator` produces a deterministic hash of the claim's key inputs (vehicle, incident, damage). If the same case signature appears in the database, the system flags it as a potential duplicate claim.

3.5 Handling Partial and Missing Data

Missing data is handled through a three-tier strategy:

1. **Explicit null:** Fields that cannot be extracted are set to `null` in `ExtractedClaimFields`. Downstream stages check for null and apply recovery strategies.
2. **Assumption recording:** When a recovery strategy is applied (e.g., vehicle mass inferred from body type), an `Assumption` object is created with `field`, `assumedValue`, `reason`,

`strategy`, `confidence` (0–100), and `stage`. All assumptions are collected and included in the final report.

3. **Degraded output:** If a critical field cannot be recovered, the affected stage produces a degraded output with reduced confidence scores. The `degradationReasons[]` array in Stage 10 lists all degradation causes for the adjuster.

The `Input Fidelity Engine (IFE)` classifies each data gap into one of four categories:

- `INSURER_DATA_GAP` — data the insurer should have provided but did not
- `DOCUMENT_LIMITATION` — data not present in the submitted documents
- `SYSTEM_EXTRACTION_FAILURE` — data present but not successfully extracted by the pipeline
- `CLAIMANT_DEFICIENCY` — data the claimant was required to provide but omitted

This classification drives the `DOE eligibility gate` — if too many gaps are classified as `SYSTEM_EXTRACTION_FAILURE`, the Decision Optimisation Engine is blocked from issuing a final recommendation until the pipeline is re-run with better documents.

---

4. Reconciliation and Decision Engine

4.1 Conflict Resolution Architecture

The KINGA pipeline produces outputs from multiple independent engines (physics, damage, cost, fraud) that may reach conflicting conclusions about the same claim. The reconciliation architecture resolves these conflicts through a layered approach:

Layer 1 — Source Truth Resolver: `sourceTruthResolver.ts` establishes the authoritative value for the four most contested fields (collision direction, vehicle speed, damage severity, repair cost) by applying a source priority hierarchy. The resolved value and its source are stored in `Stage4Output.fieldValidation`.

Layer 2 — Cross-Engine Consistency Validator: `crossEngineConsistency.ts` runs 8 named consistency checks across physics, damage, and cost outputs:

Check ID	Engines	What It Checks
`PHY_DAM_SEVERITY`	Physics + Damage	Physics-implied severity vs damage-assessed severity
`PHY_DAM_ZONE`	Physics + Damage	Impact direction vs primary damage zone
`PHY_COST_ENERGY`	Physics + Cost	Kinetic energy vs repair cost magnitude
`DAM_COST_COMPONENTS`	Damage + Cost	Damaged components vs quoted components
`DAM_FRAUD_CONSISTENCY`	Damage + Fraud	Damage pattern vs fraud narrative flags
`PHY_FRAUD_SPEED`	Physics + Fraud	Speed estimate vs claimed speed
`COST_FRAUD_DEVIATION`	Cost + Fraud	Quote deviation vs fraud risk level
`NAR_PHY_PLAUSIBILITY`	Narrative + Physics	Narrative-implied speed vs physics estimate

Each check produces an `agreement` or `conflict` record with a `severity` (`CRITICAL | SIGNIFICANT | MINOR`) and a `recommended_action`. Critical conflicts apply a `conflict_penalty` to the fraud score and set the `overallStatus` to `CONFLICTED`.

Layer 3 — Cross-Engine Consensus Engine: `crossEngineConsensus.ts` computes a final `ConsensusResult` that aggregates all engine outputs into a single `consensus_score` (0–100) and `overall_verdict`. This score is used by the Decision Optimisation Engine.

Layer 4 — Decision Optimisation Engine (DOE): `decisionOptimisationEngine.ts` applies a multi-objective scoring function:

Dimension	Weight
Cost reliability	30%
Evidence quality	25%
Data completeness	20%
Turnaround efficiency	15%
Fraud risk	10%

The DOE selects the optimal decision from `{ APPROVE, REVIEW, REJECT, NEGOTIATE, ESCALATE }` based on the weighted score, subject to hard gates:

- If `fcdiScore < 40` (high forensic degradation), the DOE cannot issue APPROVE.
- If `fraudRiskScore > 70`, the DOE cannot issue APPROVE without a manual override.

- If `decision_ready = false`, the DOE issues `PROCEED_TO_ASSESSMENT` (manual review required).

4.2 Provenance Tracking

Every conclusion in the Forensic Audit Report is linked to its source through the `EvidenceRegistry` and `ForensicExecutionLedger`:

- The `EvidenceRegistry` maps each claim section (vehicle identity, incident, damage, financial, police) to its source documents and confidence levels.
- The `ForensicExecutionLedger` records, for each stage: input hash, output snapshot, fallback used, assumptions introduced, confidence score, and model/prompt/contract versions.
- The `FELVersionSnapshot` records the exact prompt hash, model ID, and contract version for each stage, enabling deterministic replay of any historical assessment.

4.3 Adjuster Override Mechanism

Adjusters can override any constraint or recommendation through the `constraintOverridesJson` field. Each override records: `constraintId`, `accepted: boolean`, `explanation`, `overriddenBy` (user ID), and `overriddenAt` (timestamp). Overrides are included in the audit trail and trigger a re-evaluation of the DOE score.

---

5. AI and Model Usage

5.1 LLM Components

All LLM calls are routed through `server/_core/llm.ts` (`invokeLLM()`), which uses the Manus Built-in Forge API. The model is not hardcoded — the helper uses a platform-configured default. All calls are server-side only; no API key is exposed to the client.

Component	Stage	Role	Input Format	Output Format	Prompting Strategy
Document OCR	Stage 2	Verbatim text extraction from scanned pages	Image URL + OCR prompt	Plain text	Zero-shot, verbatim extraction instruction
Structured Extraction	Stage 3	Parse 50+ fields from raw text	Raw text + JSON Schema	JSON (50+ fields, nulls explicit)	JSON Schema enforcement, null-for-missing instruction
Quote Engine	Stage 3	Extract per-panel-beater quote records	Raw text + quote schema	JSON array of `ExtractedQuoteRecord`	Multi-block detection, currency normalisation
Damage Analysis	Stage 6	Per-photo component detection and severity	Image URL + damage schema	JSON `DamageAnalysisComponent[]`	Direction-aware prompt, severity taxonomy
Incident Narrative Engine	Stage 7	Causal chain, timeline, anomaly detection	Incident text + context	JSON `NarrativeAnalysis`	Chain-of-thought, structured output
Causal Reasoning Engine	Stage 7b	Causal verdict, contradiction detection	Narrative + physics + damage	JSON `CausalVerdict`	Multi-source synthesis, contradiction scoring
Vehicle Valuation	Stage 5b	Market value estimation	Make, model, year, mileage	JSON `VehicleValuation`	Market context prompt with regional pricing
Cost Narrative	Stage 9	Human-readable cost recommendation	Cost data + anomalies	JSON `costNarrative`	Evidence-grounded, recommendation-first

5.2 Vision Model Usage

The vision LLM is used in two distinct contexts:

Damage Analysis (Stage 6): Each photo is processed independently. The prompt requests a structured JSON output with a fixed schema. The direction-aware filter post-processes the output

to remove physically implausible components before they enter the damage component list.

Photo Forensics (Stage 8): A separate vision analysis pass examines each photo for manipulation indicators, EXIF anomalies, and GPS consistency. The `ai_vision_description` field captures the model's free-text description of the photo content, which is used to detect non-vehicle images (e.g., a photo of a document submitted as a damage photo).

5.3 Hallucination Mitigation

KINGA employs four layers of hallucination mitigation:

JSON Schema Enforcement: All LLM calls that produce structured data use `response_format: { type: "json_schema", json_schema: { strict: true, ... } }`. The schema uses `additionalProperties: false` and explicit `required` arrays. Fields the model cannot find must be returned as `null`, not fabricated.

Null-for-Missing Instruction: Every extraction prompt explicitly instructs the model: "If a field is not present in the document, return null. Do not infer, estimate, or fabricate values." This instruction is reinforced in the system message and the user message.

Cross-Validation Against Deterministic Outputs: LLM-produced values (e.g., damage severity, speed estimate from narrative) are cross-validated against deterministic physics calculations. Divergences exceeding 40% trigger a `HIGH_DIVERGENCE` flag and an adjuster notification.

Assumption Tracking: Any value that the LLM infers rather than directly extracts is recorded as an `Assumption` with `assumptionType: "SYSTEM_ESTIMATE"` and a confidence score below 60. These assumptions are surfaced in the Forensic Audit Report.

5.4 Determinism vs Variability Control

The physics engine (Stage 7 speed inference ensemble) is entirely deterministic — zero LLM calls. Given the same inputs, it always produces the same output. This is by design: forensic evidence must be reproducible.

LLM components introduce variability. To manage this, the system:

- Uses structured output schemas to constrain the output space.
- Records the model version and prompt hash in the FEL Version Snapshot for every assessment.
- Stores the `isReplayable` flag in the FEL snapshot, indicating whether the assessment can be reproduced exactly (true when no LLM calls were made, or when all LLM outputs are deterministically constrained by schema).

---

6. Fraud Detection and Risk Scoring

6.1 Fraud Signal Architecture

The fraud analysis engine (`stage-8-fraud.ts`) operates in three layers:

Layer 1 — Universal Indicators: Applied to every claim regardless of incident type. These include cost deviation, damage consistency, photo forensics, and claim history checks.

Layer 2 — Scenario-Aware Fraud Detection: The `scenarioFraudDetector` applies a scenario-specific fraud profile based on the `CollisionScenario` classification from Stage 5. Each scenario has a distinct set of fraud indicators with scenario-specific weights. For example:

- `rear_end_struck` claims are checked for staged accident patterns (claimant's vehicle value vs repair cost ratio, third-party cooperation).
- `single_vehicle_rollover` claims are checked for speed plausibility (rollover requires significant speed) and structural damage consistency.
- `animal_strike` claims are checked for geographic plausibility (was the accident location in a known wildlife corridor?) and species-appropriate damage patterns.

Layer 3 — Cross-Engine Consistency: The `crossEngineConsistency` validator (described in Section 4) applies a `conflict_penalty` to the fraud score for each critical cross-engine conflict.

6.2 Fraud Indicators

Indicator Category	Signals	Scoring Method
Cost Anomaly	Quote deviation from AI benchmark (%), outlier quotes, missing structural components in quote	Proportional to deviation magnitude
Damage Inconsistency	Damage pattern vs collision direction, damage severity vs physics-implied severity, components inconsistent with stated impact	Binary flags with severity weights
Photo Forensics	EXIF manipulation score, GPS inconsistency, capture datetime before accident date, image hash duplicates	Binary flags with high weights
Behavioral Patterns	Claim frequency for this claimant/vehicle, repairer history flags, rapid claim submission after policy inception	Historical lookup + threshold scoring
Physics Inconsistency	Stated speed vs physics ensemble estimate, airbag deployment vs speed estimate, structural damage vs energy calculation	Proportional to divergence magnitude
Document Anomalies	Missing police report, missing repair quote, document date inconsistencies, OCR confidence below threshold	Weighted by document importance
Third-Party Anomalies	Third-party account contradicts claimant narrative, third-party vehicle not registered, no third-party insurer	Binary flags

6.3 Scoring Methodology

The `fraudRiskScore` (0–100) is computed as a weighted sum of all active indicators. Each `FraudIndicator` has a `score` (0–100) and a `category`. The overall score is:

```
fraudRiskScore = Σ(indicator.score × indicator.weight) / Σ(indicator.weight)
```

The `fraudRiskLevel` is mapped from the score:

Score Range	Risk Level
0–20	minimal
21–40	low
41–60	medium
61–75	elevated
76–100	high

6.4 Explainability

Every fraud flag is accompanied by:

- `indicator` — a human-readable label (e.g., "Quote 47% above AI benchmark")
- `category` — the signal category (e.g., "Cost Anomaly")
- `description` — a plain-English explanation of why this is a fraud signal
- `evidence[]` — specific data points supporting the flag (e.g., "AI benchmark: $2,800; Quoted:$ 4,120; Deviation: +47%")
- `severity` — `critical | high | medium | low | advisory`

The `scenarioFraudResult.reasoning` field provides a narrative explanation of the overall fraud assessment, written for an adjuster audience.

6.5 Integration with Underwriting Decisions

The fraud score feeds directly into the Decision Optimisation Engine (Section 4.1). A `fraudRiskScore > 70` blocks the APPROVE recommendation. A `fraudRiskScore > 85` triggers an automatic ESCALATE recommendation, routing the claim to a senior risk manager.

The `claimsDecisionAuthority.ts` engine applies a final decision gate that considers fraud score, physics confidence, cost reliability, and data completeness together, producing a `ClaimsDecisionOutput` with a `decision`, `confidence`, and `decision_trace[]` (a step-by-step log of the decision logic).

---

7. Forensic Engine — Core Differentiator

7.1 Design Philosophy

The forensic physics engine is the element of KINGA that most clearly distinguishes it from vision-only damage assessment tools. Its purpose is to answer a question that photographs alone cannot: **was the claimed damage physically consistent with the stated accident?**

The engine applies classical mechanics to reconstruct the accident from available evidence. It does not require a crash test database, a black box recorder, or a physical inspection. It works from the information present in a standard insurance claim: vehicle make/model (for mass and stiffness), collision direction, damage severity, and — when available — explicit crush depth measurements.

All physics calculations are pure mathematics. They execute in under 1 millisecond on any modern CPU and produce identical results for identical inputs. This determinism is essential for forensic defensibility.

7.2 Crush Depth Inference

The crush depth (maximum permanent deformation of the vehicle structure, in metres) is the primary input to the Campbell speed formula. It is obtained through a priority cascade:

1. **Explicit document value** — if the claim form or assessor report states a crush depth (e.g., "maximum deformation: 18 cm"), this value is used directly with `HIGH` confidence.
2. **Vision-extracted value** — the Stage 6 vision LLM is prompted to estimate the visible crush depth from damage photos. This produces the M5 input with `MEDIUM` confidence.
3. **Multi-factor inference** — when neither document nor vision values are available, `inferCrushDepth()` applies a severity-based model:

```
baseline:
  cosmetic/minor  → 0.05 m
  moderate        → 0.12 m
  severe          → 0.22 m
  catastrophic    → 0.38 m

additive modifiers:
  component count > 3   : +0.01 m per component (cap +0.08 m)
  structural damage      : +0.06 m
  damage area > 0.2 m²   : +0.008 m per 0.1 m² (cap +0.04 m)
  airbag deployment      : floor raised to 0.15 m

result clamped to [0.04 m, 0.55 m]
```

The `massTier` and crush depth source are both recorded in the Forensic Audit Report so the adjuster understands the basis of the physics calculation.

7.3 Five-Method Speed Inference Ensemble

The `runSpeedInferenceEnsemble()` function in `speedInferenceEnsemble.ts` runs up to five independent methods in parallel:

M1 — Campbell's Stiffness Formula (PRIMARY)

```
V = √(2 × k × C² / m)

where:
  k = vehicle structural stiffness (kN/m), from body-type lookup table
  C = crush depth (m)
  m = vehicle mass (kg)
```

Stiffness values are derived from NHTSA crash test data and Campbell (1974):

Body Type	Stiffness (kN/m)
Compact	800
Sedan	1,000
SUV	1,200
Pickup	1,350
Truck	1,400
Bus	1,600

Accident-type multipliers adjust for energy distribution: frontal (1.00), rear (0.90), side (1.10), rollover (1.30). Structural damage adds 12% to the speed estimate. Airbag deployment floors the estimate at 22 km/h.

Confidence: `HIGH` (0.90 weight) when crush depth is document-stated; `MEDIUM` (0.60 weight) when inferred.

M2 — Energy-Momentum Balance (DISABLED)

This method used repair cost as a proxy for deformation energy (Strother et al. 1986, SAE 860924). It has been deliberately disabled because the 1986 US-market cost/energy correlation does not transfer reliably to other markets or time periods. The method slot is preserved in the output schema for UI consistency, but it always returns `ran: false`.

M3 — Impulse-Momentum Method

```
V = (F_contact × Δt) / m

where:
  F_contact = contact_pressure × damage_area (using 4 MPa, SAE 930899)
  Δt = 2C / V (iterative — solved from initial rough estimate)
  m = vehicle mass (kg)
```

Confidence: `MEDIUM` (0.40 weight) when damage area is available from Stage 6.

M4 — Deployment Threshold (Hard Lower Bound)

FMVSS 208 defines the minimum speed at which frontal airbags deploy: 20–30 km/h equivalent barrier speed. This method does not produce a point estimate — it establishes a hard lower bound:

- Airbag deployment confirmed → $V \geq 20$ km/h (typical: 25–35 km/h)
- Seatbelt pretensioner only → $V \geq 15$ km/h

M4 is excluded from the weighted mean but used to floor the consensus estimate.

M5 — Vision Deformation Estimate

Applies the same Campbell formula as M1, but uses the crush depth extracted by the Stage 6 vision LLM from damage photos rather than the document-stated value. This provides an independent cross-check on the document value.

Confidence: `MEDIUM` (0.70 weight) when vision depth is available.

7.4 Consensus Algorithm

```
1. Run M1, M2, M3, M4, M5 in parallel.
2. Collect point estimates (M1, M3, M5) — exclude lower-bound-only methods (M4).
3. Compute initial weighted mean.
4. Outlier rejection: methods deviating > 2σ from initial mean are down-weighted by
50%.
5. Compute final weighted consensus speed.
6. Apply M4 lower bound floor.
7. 90% confidence interval: ± (weighted_std_dev × 1.645).
8. Cross-validation: if any two estimates differ by > 40%, set HIGH_DIVERGENCE flag.
9. Overall confidence: HIGH if ≥ 2 HIGH-confidence methods; MEDIUM if ≥ 2 methods ran;
LOW otherwise.
```

7.5 Downstream Physics Calculations

Given the consensus speed, the engine computes:

```
KE = ½ × m × v²                          (kinetic energy, Joules)
E_dissipated = KE × 0.6                  (60% absorbed in deformation)
a = v² / (2 × crush_depth)              (deceleration, m/s²)
F = m × a                                (impact force, Newtons)
g_force = a / 9.81                       (deceleration in g)
delta_V = v × 0.6                        (approximate velocity change)
```

The `latentDamageProbability` map estimates the probability of hidden damage in five systems (engine, transmission, suspension, frame, electrical) based on impact force and collision direction.

7.6 Damage Pattern Validation

`damagePatternValidationEngine.ts` cross-checks the damage component list against the expected damage pattern for the stated collision scenario. For a frontal impact, the expected primary damage zone is the front (bumper, grille, bonnet, headlights, radiator). If the reported damage is concentrated in the rear zone, the validator flags a `PATTERN_MISMATCH` with a `plausibility_score` below 50.

This validation is scenario-specific — animal strikes, rollovers, and side impacts each have distinct expected damage patterns.

---

8. System Reliability and Failure Handling

8.1 Never-Halt Design

The pipeline's most important reliability property is that it **never halts**. Every stage is wrapped in a `try/catch` block. Every catch handler produces a documented fallback output. The orchestrator always returns a `Stage10Output`, even if every stage failed — in that case, the report contains only the claim's database fields with all confidence scores at their minimum values and a full list of degradation reasons.

8.2 Stage Timeout Enforcement

`pipelineContractRegistry.ts` defines a time budget for each stage. `runWithTimeout(stageKey, fn)` wraps every stage call. If a stage exceeds its budget, a `StageTimeoutError` is thrown and caught by the orchestrator's per-stage catch handler. The timeout is recorded as a `stage_timeout` assumption with `confidence: 5`.

8.3 Fallback Strategies

Stage	Fallback Strategy
Stage 1 (Ingestion)	Empty document set — all downstream stages use claim DB fields
Stage 2 (OCR)	Empty text set — Stage 3 uses claim DB fields directly
Stage 3 (Extraction)	All fields null — Stage 4 uses claim DB fields
Stage 4 (Validation)	Raw Stage 3 output used directly
Stage 5 (Assembly)	Minimal ClaimRecord from DB fields
Stage 6 (Damage)	`buildDamageFallback()` — text-based damage list, severity 50, no photos
Stage 7 (Physics)	`estimatePhysicsFromDamage()` — simplified KE formula, or `SKIPPED_NO_SPEED`
Stage 8 (Fraud)	`buildFraudFallback()` — minimal fraud output, score 50, no indicators
Stage 9 (Cost)	`buildCostFallback()` — document quote total, LOW reliability
Stage 10 (Report)	Minimal report with all available data and full degradation reasons list

8.4 Pipeline State Machine

`pipelineStateMachine.ts` tracks the pipeline's execution state through transitions:

```
INITIALISED → RUNNING → COMPLETED
                    ↓
              FLAGGED_EXCEPTION (non-fatal — pipeline continues)
                    ↓
              COMPLETED_WITH_EXCEPTIONS
```

`CRITICAL_STAGES` (Stages 1, 2, 3, 5) trigger `FLAGGED_EXCEPTION` on failure. Non-critical stage failures are logged but do not change the state machine.

`runAnomalySentinels()` runs after all stages complete and checks for anomalous patterns (e.g., all stages succeeded but confidence score is below 20, suggesting a systematic data quality issue).

8.5 Forensic Confidence Degradation Index (FCDI)

`forensicCDI.ts` computes the FCDI (0–100) from:

- Fallback count (each fallback reduces the score)
- Timeout count
- Assumption count (weighted by confidence level)
- Low-confidence stage count (stages with output confidence below 40)
- Domain penalty (applied when automotive domain corrections were needed)

FCDI 100 = fully reliable (all stages succeeded, no assumptions, no fallbacks). FCDI 0 = maximally degraded (most stages failed or fell back).

The FCDI is stored in `ai_assessments.fcdiScore` and displayed prominently in the adjuster dashboard. It gates the DOE — a FCDI below 40 blocks the APPROVE recommendation.

8.6 Logging and Observability

Every stage calls `ctx.log(stageName, message)` for structured logging. The `pipelineRunSummary` JSON column stores per-stage health data (status, duration, error message, assumption count, recovery action count) for every pipeline run. This enables:

- Dashboard monitoring of pipeline health across all claims
- Detection of systemic failures (e.g., vision API unavailable — all Stage 6 runs degraded)
- Per-claim debugging without re-running the pipeline

The `imageAnalysisTotalCount`, `imageAnalysisSuccessCount`, `imageAnalysisFailedCount`, and `imageAnalysisSuccessRate` columns track vision API success rates per assessment run, enabling automated alerting when the success rate drops below a threshold.

---

9. UI and Reporting Layer

9.1 Adjuster Dashboard

The adjuster dashboard is a React 19 single-page application served at the platform's root URL. It is built on the `DashboardLayout` component with a persistent sidebar navigation. Key panels:

Claim List: Paginated list of all claims with status, fraud risk level, recommendation, and pipeline health indicator. Filterable by status, risk level, date range, and tenant.

Claim Detail: Full claim view with tabs for:

- **Summary** — claim identity, vehicle, incident overview, and pipeline recommendation
- **Forensic Audit Report** — the full structured report rendered as a multi-section document
- **Physics Reconstruction** — speed inference ensemble table with per-method results, confidence intervals, and divergence flags
- **Damage Analysis** — per-photo results, component list with severity scores, damage zone map
- **Cost Intelligence** — AI benchmark vs quoted cost, parts reconciliation table, negotiation guidance
- **Fraud Risk** — fraud score gauge, indicator list with evidence citations, scenario analysis
- **Evidence Trace** — pipeline run summary, assumption registry, FEL snapshot

Admin Panel: Tenant management, user management, pipeline health monitoring, and calibration controls.

9.2 Forensic Audit Report

The Forensic Audit Report is the primary deliverable — a structured document that an adjuster can use to make a settlement decision or escalate a claim. It is designed around the principle of **clarity over complexity**: every conclusion is accompanied by its evidence basis and confidence level.

Report sections:

1. Claim Summary (identity, vehicle, incident, recommendation)
2. Speed-Physics Evidence (ensemble table, consensus speed, confidence interval)
3. Damage Analysis (component list, zone map, structural damage flag)

4. Cost Intelligence (benchmark, deviation, reconciliation, negotiation guidance)
5. Fraud Risk Assessment (score, indicators, scenario analysis)
6. Evidence Quality (FCDI score, assumption registry, missing documents)
7. Supporting Images (damage photos with per-photo analysis overlays)

9.3 Decision-Support Features

Recommendation Badge: The pipeline recommendation (APPROVE / REVIEW / REJECT / NEGOTIATE / ESCALATE) is displayed prominently with a colour-coded badge. The `decision_trace[]` provides a step-by-step log of the decision logic for transparency.

Constraint Override UI: Adjusters can override any constraint or recommendation with a mandatory explanation. Overrides are logged and included in the audit trail.

Reanalysis Trigger: Adjusters can trigger a pipeline reanalysis at any time (e.g., after uploading additional documents). The new assessment is stored as a versioned record alongside the original.

Negotiation Guidance Panel: When the recommendation is NEGOTIATE, the UI displays the target price, floor, ceiling, and per-panel-beater reduction recommendations from the Stage 9c negotiation guidance package.

---

10. Integrations and Extensibility

10.1 API Architecture

The platform exposes a tRPC API at `/api/trpc`. All procedures are defined in `server/routers.ts` and its sub-routers. The tRPC layer provides end-to-end type safety — the TypeScript types defined in `server/pipeline-v2/types.ts` flow directly to the React frontend without a separate API contract file.

External systems can integrate via the REST-compatible tRPC HTTP adapter. Authentication uses JWT session cookies issued by the Manus OAuth flow.

10.2 Multi-Tenancy

The platform is fully multi-tenant. Every database table includes a `tenantId` column. The `tenants` table stores per-tenant configuration:

Configuration	Description
`labourRateUsdPerHour`	Overrides the regional default labour rate
`paintCostPerPanelUsd`	Overrides the global paint cost default
`currencyCode`	Display currency (e.g., ZAR, USD)
`intakeEscalationEnabled`	Whether to auto-escalate stale intake claims
`intakeEscalationHours`	Hours before auto-escalation triggers

Tenant isolation is enforced at the database query level — all queries include a `WHERE tenantId = ?` clause. Cross-tenant data access is not possible through the application layer.

10.3 Document Intake Channels

Documents can be submitted through:

- **Web Upload:** Direct file upload through the adjuster dashboard (PDF, images, Word documents)
- **API:** POST to `/api/trpc/claims.uploadDocuments` with multipart form data
- **WhatsApp Integration (planned):** WhatsApp Business API webhook to receive photos and documents from claimants directly, reducing intake friction

10.4 External Data Sources

Source	Usage	Integration Method
AutoTrader ZA	Vehicle market valuation	LLM-mediated web search + structured extraction
ZINARA (Zimbabwe)	Vehicle registration verification	API (planned)
ZRP (Zimbabwe Republic Police)	Police report verification	Manual reference check (automated API planned)
IPEC (Zimbabwe)	Regulatory compliance data	Reference data
Parts price databases	Component cost benchmarking	Regional rate tables (updated quarterly)

10.5 Scalability Architecture

The platform is deployed on Google Cloud Run (via Manus hosting), which provides automatic horizontal scaling based on request load. The stateless Express server can scale to multiple instances without coordination.

The MySQL/TiDB database is the primary scalability bottleneck. The schema includes composite indexes on the most common query patterns:

- `idx_ai_assessments_claim_id` — claim detail lookups
- `idx_ai_claim_confidence` — confidence-filtered claim lists
- `idx_ai_tenant_fraud` — tenant-scoped fraud risk queries

The pipeline itself is CPU-bound for physics calculations and I/O-bound for LLM calls. LLM calls are the dominant latency contributor — a standard 5-document claim with 3 damage photos requires approximately 8–12 LLM calls across all stages, with a total latency of 60–120 seconds depending on model response times.

---

11. Security and Governance

11.1 Data Privacy

All claim documents and damage photos are stored in S3 with non-enumerable paths (random suffixes on all file keys). The S3 bucket is not publicly listable. Pre-signed URLs are used for all document access.

Personally identifiable information (PII) — claimant names, driver licence numbers, ID numbers — is stored in the database but is subject to the `anonymizationAuditLog` process. The `anonymizationAuditLog` table records every anonymisation operation with its status (`success | withheld_k_anonymity | withheld_pii_detected | withheld_tenant_opt_out`) and the quasi-identifier hash used to verify k-anonymity compliance.

11.2 Audit Trails

The platform maintains three independent audit trails:

Forensic Execution Ledger (FEL): Per-stage record of input hash, output snapshot, model version, prompt hash, fallback used, and assumptions introduced. Designed for court-grade audit requirements.

FEL Version Snapshot: Per-stage record of the exact code version, prompt version, and model ID used for each assessment. Enables deterministic replay of historical assessments.

Access Denial Log: Every unauthorised access attempt is recorded in `access_denial_log` with user ID, attempted route, user role, insurer role, tenant ID, denial reason, IP address, and user agent.

11.3 Role-Based Access Control

The platform implements a three-tier RBAC model:

Role	Access
`owner`	Full platform access, tenant management, user management
`admin`	Full claim access within their tenant, user management within tenant
`user`	Claim submission and viewing within their tenant; no admin functions

Insurer-specific roles (`assessor`, `risk_surveyor`, `risk_manager`) are stored in a separate `insurerRole` field and control access to the approval workflow. The three-level approval chain (assessor → risk surveyor → risk manager) is enforced by the `approvalWorkflow` table.

11.4 Compliance Considerations

The platform is designed for compliance with:

- **Zimbabwe Data Protection Act (2021):** PII handling, consent, and data subject rights
- **IPEC Motor Insurance Regulations:** Claim processing standards and documentation requirements
- **GDPR principles (for cross-border data):** Data minimisation, purpose limitation, and audit trail requirements

The `assumptionRegistryJson` and `forensicExecutionLedgerJson` columns provide the documentation required to explain any AI-assisted decision to a regulator or court.

---

12. Performance and Scalability

12.1 Expected Throughput

A single Cloud Run instance (2 vCPU, 4 GB RAM) can process approximately 20–30 claims per hour under normal conditions. The primary bottleneck is LLM API latency, not compute.

Claim Type	Expected Pipeline Duration
Simple (1 document, no photos)	30–60 seconds
Standard (3–5 documents, 3–5 photos)	90–180 seconds
Complex (10+ documents, 10+ photos, multiple quotes)	3–6 minutes

12.2 Parallelisation Strategy

Within a single claim, the pipeline is currently sequential. Stages 6–9 are logically independent given the `ClaimRecord` and are candidates for parallel execution. The `evidenceTrace.parallelStages` field is designed to document concurrent execution when it is implemented.

Across claims, the platform is fully parallel — each claim runs in its own pipeline invocation with no shared state.

12.3 LLM Cost Management

LLM calls are the dominant cost driver. The platform manages costs through:

- **Photo budget:** A maximum of 5 photos are sent to the full vision analysis per claim. Additional photos are deferred (`SKIPPED_BUDGET`) unless the adjuster explicitly requests full analysis.
- **Image Intelligence Layer:** A pre-filter scores each photo for damage likelihood before the full vision analysis, avoiding LLM calls on non-damage images.
- **Structured output schemas:** JSON Schema enforcement reduces the need for retry calls due to malformed output.
- **Stage 2 OCR selectivity:** Only scanned (non-native) PDF pages are sent to the vision OCR model. Native PDF text is extracted without LLM calls.

12.4 Database Performance

The `ai_assessments` table is the largest table in the schema, with 30+ JSON columns storing stage outputs. JSON column queries are not indexed. The platform mitigates this by:

- Storing scalar summary values (fraud score, recommendation, confidence score) as indexed columns for fast filtering.
- Using the JSON columns only for full report rendering, not for list queries.
- The `pipelineRunSummary` column provides per-stage health data without requiring JSON parsing of individual stage output columns.

---

13. Limitations and Future Improvements

13.1 Current Limitations

Vision Analysis Dependency on Photo Quality: The Stage 6 damage analysis is only as good as the submitted photos. Blurry, poorly lit, or obstructed photos produce low-confidence outputs. The `imageConfidenceScore` and `photosProcessed` fields surface this limitation to the adjuster, but the system cannot compensate for fundamentally poor photo quality.

M2 (Energy-Momentum) Disabled: The repair cost-to-energy correlation (Strother et al. 1986) has been disabled because the 1986 US-market calibration does not transfer to Zimbabwe or South Africa. A market-specific calibration dataset would be required to re-enable this method.

Vehicle Mass Inference Accuracy: For vehicles not in the model-specific lookup table, mass is inferred from body type class averages. This introduces a ±15–20% uncertainty in all physics calculations. The `massTier` field surfaces this uncertainty, but it cannot be eliminated without a comprehensive vehicle database.

No Real-Time External Data: Vehicle registration verification (ZINARA), police report verification (ZRP), and parts price feeds are not yet integrated as live APIs. The system uses LLM-mediated web search and static rate tables as proxies.

Sequential Pipeline: Stages 6–9 run sequentially. For complex claims with many photos and multiple quotes, this contributes to longer processing times. Parallel execution of independent stages is a planned improvement.

LLM Non-Determinism: LLM components introduce variability across runs. While JSON Schema enforcement constrains the output space, two runs of the same claim may produce slightly different narrative analyses or damage descriptions. The `isReplayable` flag in the FEL snapshot documents this limitation.

13.2 Planned Enhancements

M5 Vision Crush Depth Integration: The Stage 6 vision LLM prompt will be updated to explicitly extract `crushDepthM` as a structured field, enabling M5 to contribute to the speed inference ensemble for all claims with photos.

Live ZINARA Integration: Direct API integration with the Zimbabwe National Road Administration for real-time vehicle registration verification.

WhatsApp Intake: WhatsApp Business API integration to allow claimants to submit photos and documents directly via WhatsApp, reducing intake friction and improving photo quality.

Calibration Feedback Loop: The `validatedOutcomeRecorder` and `calibrationDriftDetector` are designed to feed adjuster decisions back into the system's calibration. When an adjuster

overrides a recommendation, the override is recorded and used to recalibrate the fraud scoring weights and cost benchmark rates over time.

Parallel Stage Execution: Stages 6, 8, and 9 can be parallelised once Stage 7 output is available. This would reduce processing time for complex claims by approximately 40%.

Structured Parts Database: A Zimbabwe/South Africa-specific parts price database, updated monthly from supplier catalogues, would replace the current LLM-estimated component costs with deterministic lookups.

13.3 Research Opportunities

Accident Reconstruction from Video: Dashcam footage, when available, could provide direct crush depth measurements and impact speed estimates through optical flow analysis, replacing inferred values with measured ones.

Fleet Risk Scoring: Aggregating physics and damage patterns across a fleet owner's claims could identify high-risk drivers, routes, and vehicle models — enabling proactive risk management rather than reactive claims processing.

Fraud Network Detection: Graph analysis of claimant, repairer, and assessor relationships across claims could identify coordinated fraud rings that are invisible at the individual claim level.

---

14. Executive Summary

What KINGA Is

KINGA AutoVerify AI is a modular, self-healing AI platform that automates the forensic analysis of motor insurance claims. It ingests raw claim documents — PDFs, scanned forms, repair quotes, police reports, and damage photographs — and produces a structured Forensic Audit Report within minutes. The report covers accident physics reconstruction, multi-source damage analysis, repair cost benchmarking, fraud risk scoring, and a decision recommendation, all with full auditability and explainability.

Why It Is Powerful

KINGA's power derives from the combination of three capabilities that no existing tool in the African insurance market provides together:

Physics-based forensic reconstruction. The five-method speed inference ensemble (Campbell, Impulse, Vision Deformation, Deployment Threshold) applies classical mechanics to reconstruct the accident from available evidence. This produces a quantified, defensible answer to the

question “was this damage physically consistent with the stated accident?” — a question that photographs and adjuster intuition cannot reliably answer.

Self-healing pipeline architecture. The pipeline never halts. Every stage has a documented fallback. Every assumption is recorded, classified, and surfaced to the adjuster. A claim with poor documentation produces a degraded report with low confidence scores — not a processing failure. This means the system is deployable in real-world conditions where document quality is variable and unpredictable.

Full auditability. The Forensic Execution Ledger records every stage’s input hash, output snapshot, model version, and assumptions. The FEL Version Snapshot enables deterministic replay of any historical assessment. Every fraud flag is accompanied by specific evidence citations. This level of auditability is a prerequisite for regulatory acceptance and legal defensibility.

Why It Is Defensible

KINGA’s conclusions are defensible because they are grounded in established engineering standards:

- Campbell (1974) stiffness formula — the standard method for crush-based speed estimation in accident reconstruction
- FMVSS 208 airbag deployment thresholds — a regulatory standard with known physical bounds
- SAE 930899 impulse-momentum method — a peer-reviewed engineering method
- NHTSA crash test correlation data — the empirical basis for vehicle stiffness values

The physics engine produces identical results for identical inputs. The assumption registry documents every deviation from direct measurement. The cross-engine consistency validator flags internal contradictions before they reach the adjuster. These properties make KINGA’s outputs suitable for use in claim disputes, regulatory reviews, and legal proceedings.

Why It Is Investable

The Zimbabwe motor insurance market processes approximately 513,000 active policies annually, with H1 2025 gross written premium of $47.05 million. Manual assessment costs 25–50 per claim. KINGA reduces this to 5–$12 per claim while improving consistency and fraud detection.

The platform’s data flywheel compounds its value over time. Every processed claim adds to the calibration dataset for cost benchmarking, fraud signal weights, and physics model accuracy. The `validatedOutcomeRecorder` feeds adjuster decisions back into the system’s calibration, creating a learning loop that improves accuracy with scale.

The architecture is designed for regional expansion. Multi-tenancy, configurable per-tenant rates, multi-currency support, and scenario-specific fraud profiles mean that adding a new insurer or a new market (South Africa, Zambia, Kenya) requires configuration, not re-engineering.

At $8 per claim for a sector-wide ICZ rate, a$10410,000 in annual recurring revenue from a single market — before accounting for the cost savings delivered to insurers, the fraud prevented, and the premium reduction enabled by better risk data.

This document was prepared from the KINGA AutoVerify AI codebase as of April 2026. All technical specifications are derived from the production source code in `server/pipeline-v2/`*. Physics constants and engineering references are cited inline.*